Routing and Load Balancing

Deep Dive into SGLang — Chapter 25

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns "pick a worker" into "run a service in front of a worker fleet."

- A request names a model; the gateway must decide which backend can accept it *now*.
- Routing is a control-plane decision made *before* any Python scheduler or CUDA kernel sees the request.
- In SGLang the gateway is a separate Rust subsystem — traits, registries, atomics, and destructor-based cleanup.

Goal: by the end you can look at a routing decision and say which worker *may* serve it, which one *should*, and who pays if a worker fails.

Roadmap

Admission before scoring

The gateway's moving parts

Failure isolation and lifetime

Admission before scoring

Two facts routing must never conflate

- **Eligibility** says a worker *may* serve the request: right model, right role, healthy, breaker closed.
- **Scoring** chooses *among* eligible workers by load or locality.

$$b^* = \arg\min_{b \in \mathcal{E}(r)} S(b, r).$$

A backend can be the fastest and still be ineligible. Load never repairs an invalid candidate.

From concept to code: eligibility, then least load

The whole “admit before score” idea is one Rust method.

from `sgl-model-gateway/src/routers/openai/router.rs`

```
fn find_best_worker_for_model(&self, model_id: &str) -> Option<Arc<dyn Worker>> {
    self.worker_registry
        .get_workers_filtered(None, None, None, Some(RuntimeType::External), true)
        .into_iter()
        .filter(|w| w.supports_model(model_id) && w.circuit_breaker().can_execute())
        .min_by_key(|w| w.load())
}
```

Read the pipeline: filter external workers, keep only those that `supports_model` and whose breaker `can_execute`, then `min_by_key` on `load()`. Eligibility is the `filter`; scoring is the `min_by_key`.

The gateway's moving parts

Three owners a contributor must not collapse

Registry owns *which workers are candidates* — model index, role, health, and per-model hash rings.

Policy owns *how to choose* among candidates — returns an *index*, never legalizes a worker.

Router owns *protocol side effects* — transform, forward, retry, and circuit-breaker feedback.

One request crosses all three, in order. Confusing them is the source of most routing bugs.

The policy family: which signal dominates?

Policy	Main signal	Main risk
Random / round-robin	uniform / turn order	ignores cost and locality
Power-of-two	two sampled workers, comparable load	misses global minimum
Cache-aware	approximate text prefix tree	hot prefix overloads a worker
Prefix hash	bounded token-prefix hash + ring	coarse; needs token hints
Consistent hashing	stable key over the ring	hot keys dominate balance
Manual / bucket	header key / size buckets	hotspots; length is a proxy

Eight rows, one contract: each is the *same* eligibility-then-score decision; they only differ in which post-eligibility signal wins.

Cache-aware routing on one line of algebra

A per-model text tree predicts reuse, but balance can override it. Route to the owning worker only when the pool is balanced:

$$\left(\ell_{\max} - \ell_{\min} > \tau_{\text{abs}}\right) \wedge \left(\ell_{\max} > \tau_{\text{rel}} \cdot \ell_{\min}\right) \Rightarrow \text{imbalanced} \Rightarrow \text{least load.}$$

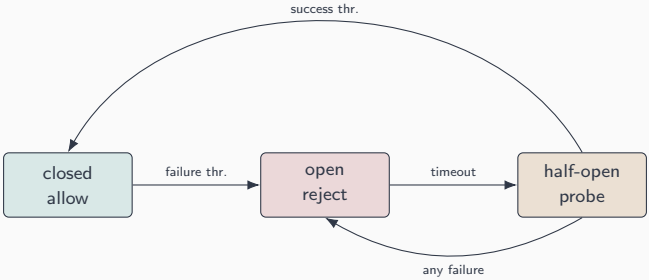
Otherwise use the tree, keeping locality only when the match rate clears a threshold:

$$\text{match rate} = \frac{\text{matched chars}}{\text{request chars}} > \tau_{\text{cache}} \Rightarrow \text{route to owner.}$$

Locality wins only when the pool is balanced *and* the prefix match is strong.

Failure isolation and lifetime

Failure becomes routing state: the circuit breaker



- An open breaker removes a worker from the eligible set — failure isolation costs capacity on purpose.
- An atomic compare-and-swap admits exactly *one* half-open probe, preventing a recovery stampede.

From concept to code: load charged for the request lifetime

In-flight load is an RAII guard: construct charges, destructor releases.

from **sgl-model-gateway/src/core/worker.rs**

```
impl WorkerLoadGuard {
    pub fn new(worker: Arc<dyn Worker>, headers: Option<&http::HeaderMap>) -> Self {
        worker.increment_load();           // n_w <- n_w + 1
        // ... remember optional routing key ...
    }
}
impl Drop for WorkerLoadGuard {
    fn drop(&mut self) { self.worker.decrement_load(); /* n_w <- n_w - 1 */ }
}
```

A response-body wrapper keeps the guard alive until the stream ends — so a 120-token stream stays charged after the router function has returned.

Three routing invariants

1. **Eligible-backend routing:** the worker scored, forwarded to, charged, and reported is the *same* eligible backend.
2. **Separated signal accounts:** request-lifetime load, sampled token/queue load, and mesh cache state each keep their own unit and freshness — no one is proof of another.
3. **Request-lifetime load guard:** a local increment stays charged for exactly the lifetime of the forwarded request or stream.

These three are the chapter's spine — everything else is a policy choosing where to sit in the locality-versus-balance tradeoff.

What to carry forward

- Routing is an **online control-plane decision**: eligibility first, scoring second, always.
- The gateway is one **Rust boundary** with three owners: registry, policy, router.
- **Policies** differ only in which post-eligibility signal wins; **PD routing** composes two of them into a pair.
- **Circuit breakers** turn failure into eligibility state; **load guards** keep accounting honest across streams.
- Every concept has a **home in** `sgl-model-gateway/`.

Next: Section 1 makes the eligible set precise — admission before any worker is scored.